



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт кибербезопасности и цифровых технологий (ИКБ)

КБ-2 «Информационно-аналитические системы кибербезопасности»

ОТЧЕТ О ВЫПОЛНЕНИИ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ №6
В РАМКАХ ДИСЦИПЛИНЫ «ТЕХНОЛОГИИ
ХРАНЕНИЯ В СИСТЕМАХ КИБЕРБЕЗОПАСНОСТИ»

Выполнил:

Студент 3-ого курса

Учебной группы БИСО-02-22

Зубарев В.С.

Москва 2024

Создайте каталог для нового проекта и сформируйте файл `docker-compose.yml` для разворачивания MinIO (<https://hub.docker.com/r/minio/minio>) в режиме SingleNode Single-Drive (1 сервер, 1 диск).

Файл `docker-compose.yml`

```
version: '3.7'
```

```
services:
```

```
  minio:
```

```
    container_name: minio_zvs
```

```
    restart: always
```

```
    image: minio/minio:latest
```

```
    ports:
```

```
      - "12005:9000"
```

```
      - "13005:9090"
```

```
    environment:
```

```
      MINIO_ROOT_USER: admin_zvs
```

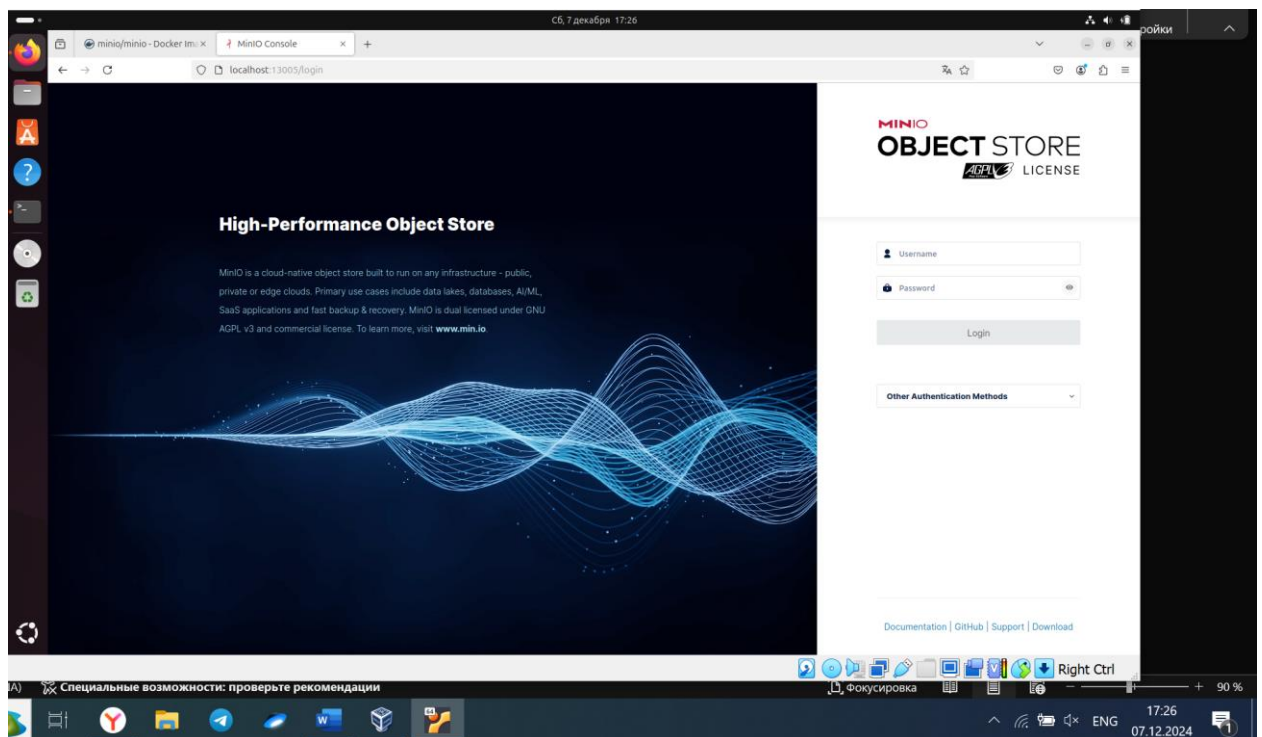
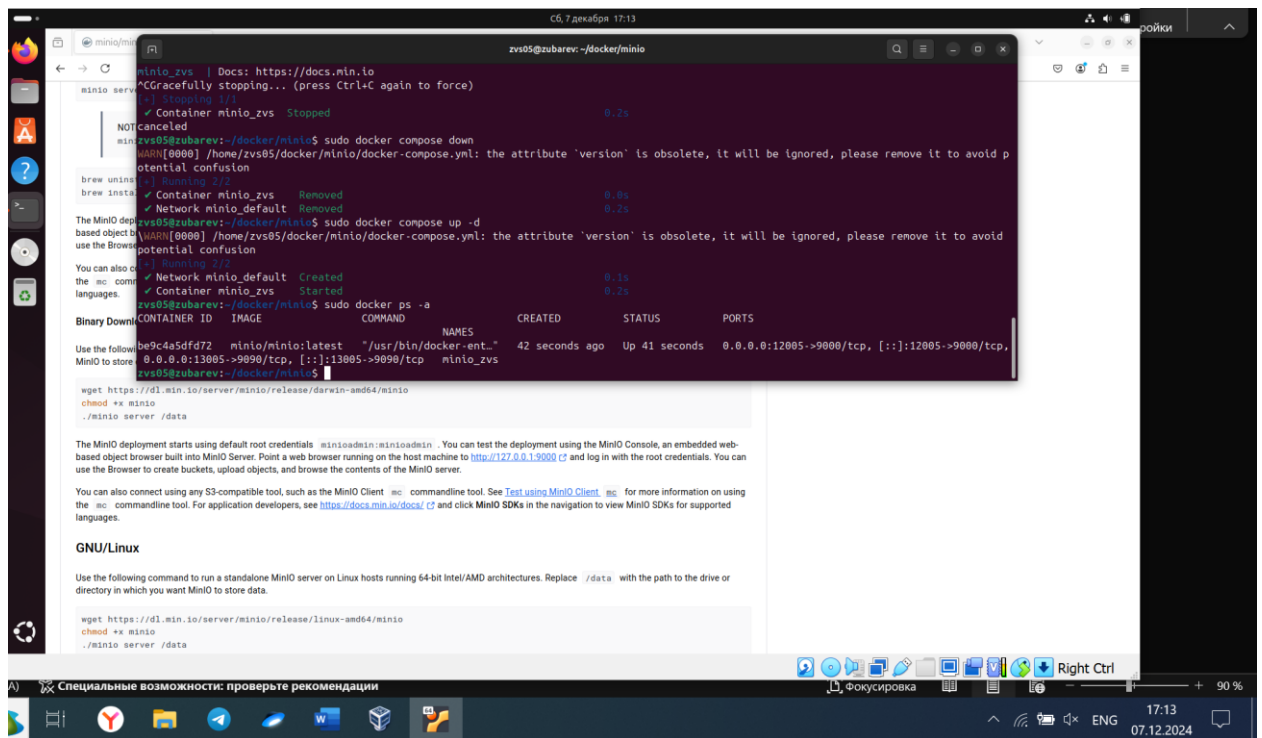
```
      MINIO_ROOT_PASSWORD: 12345678
```

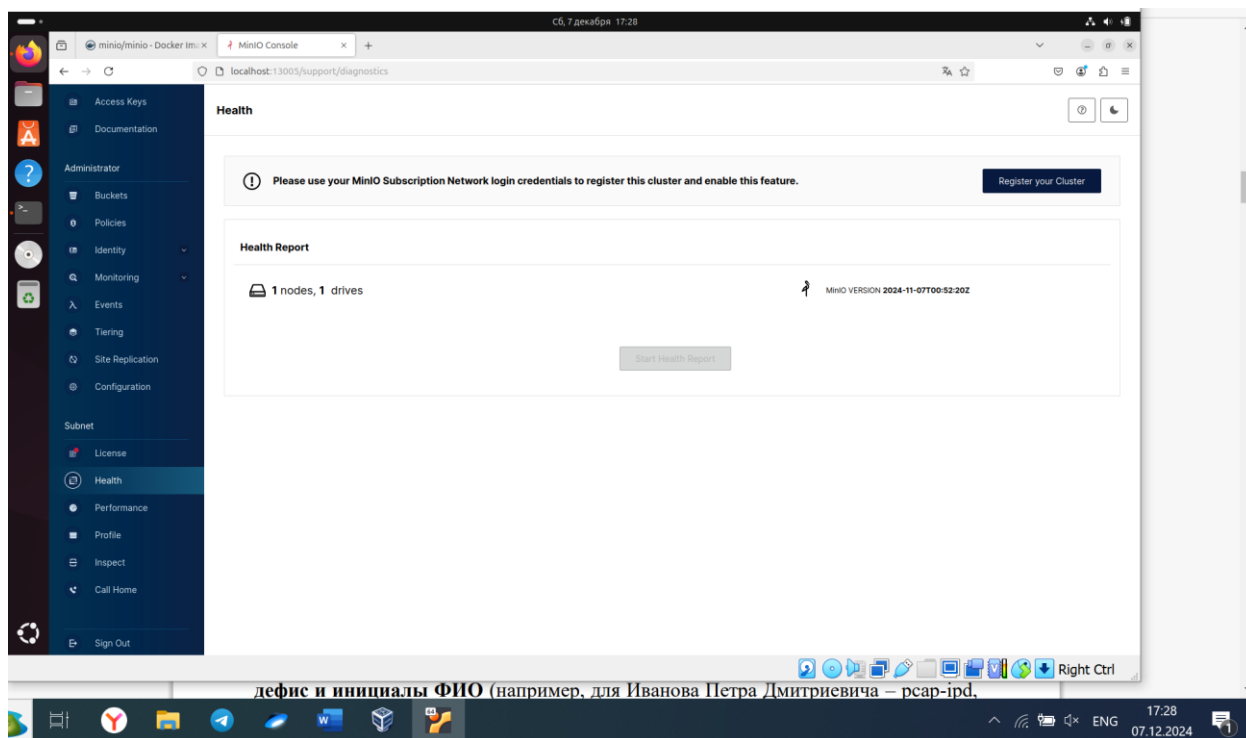
```
    command: server /data --console-address ":9090"
```

```
    volumes:
```

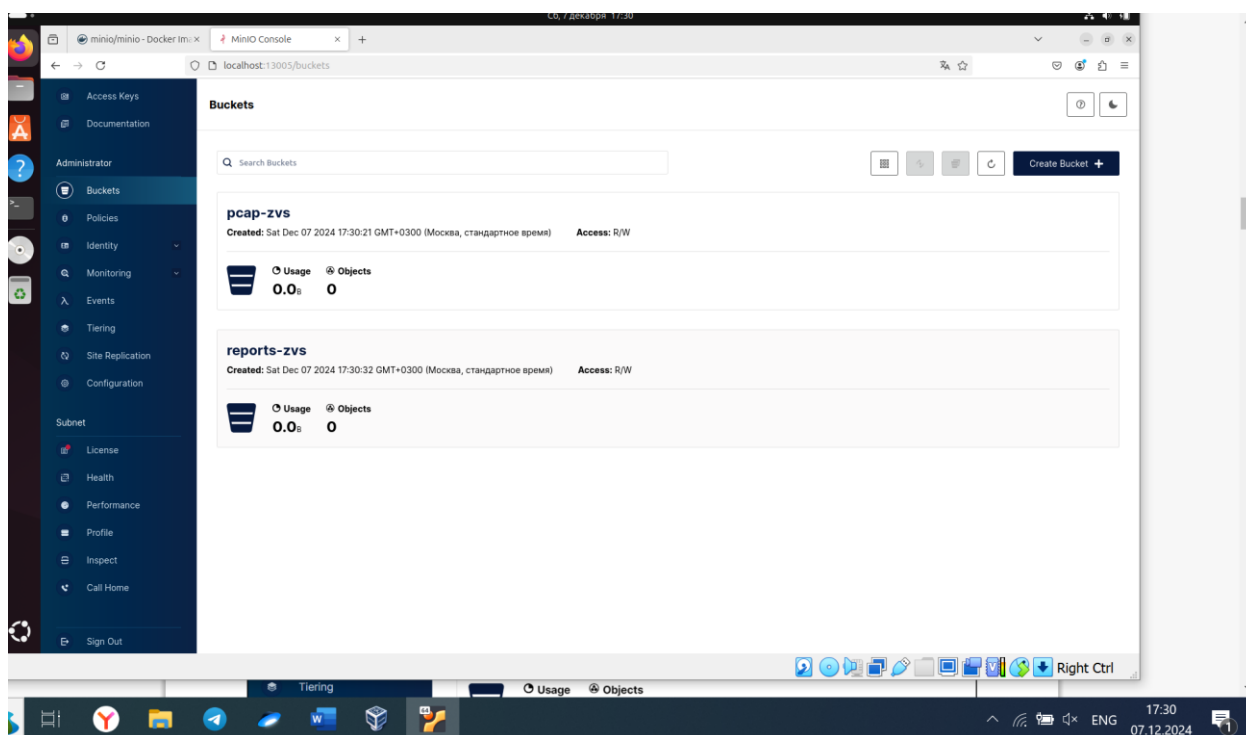
```
      - ./data:/data
```

```
      - ./config:/root/.minio
```

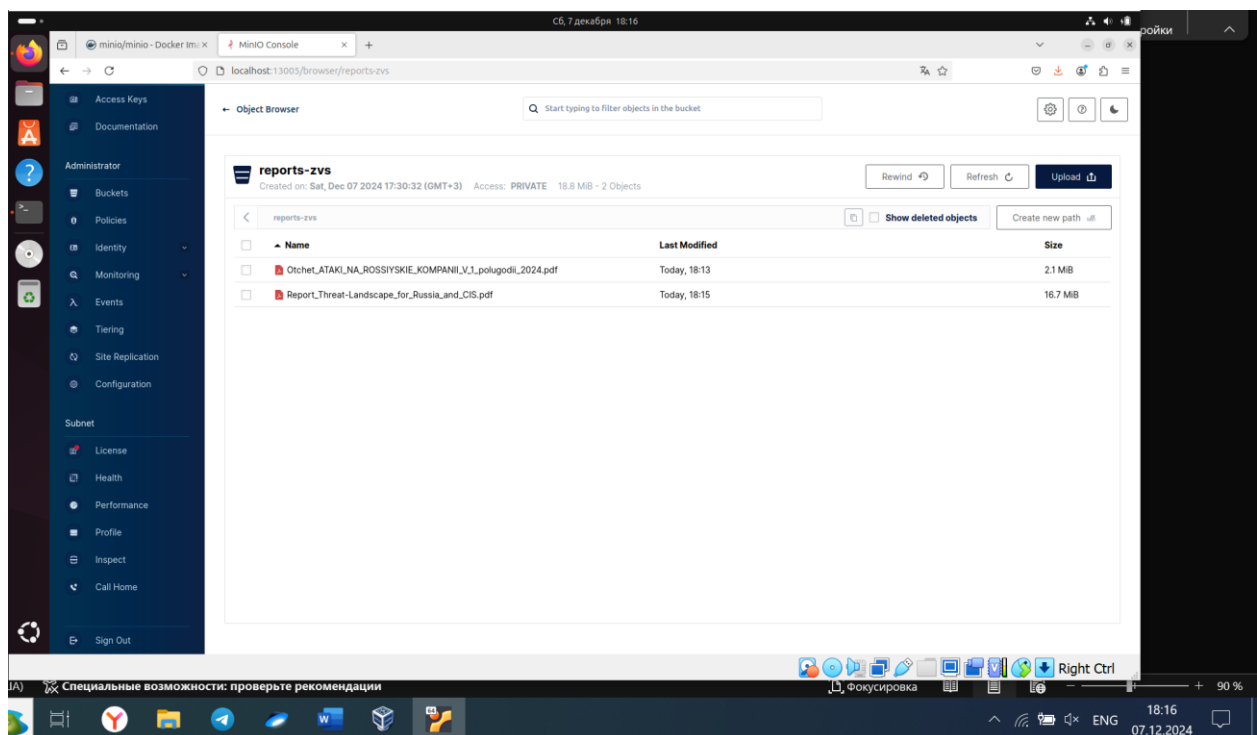
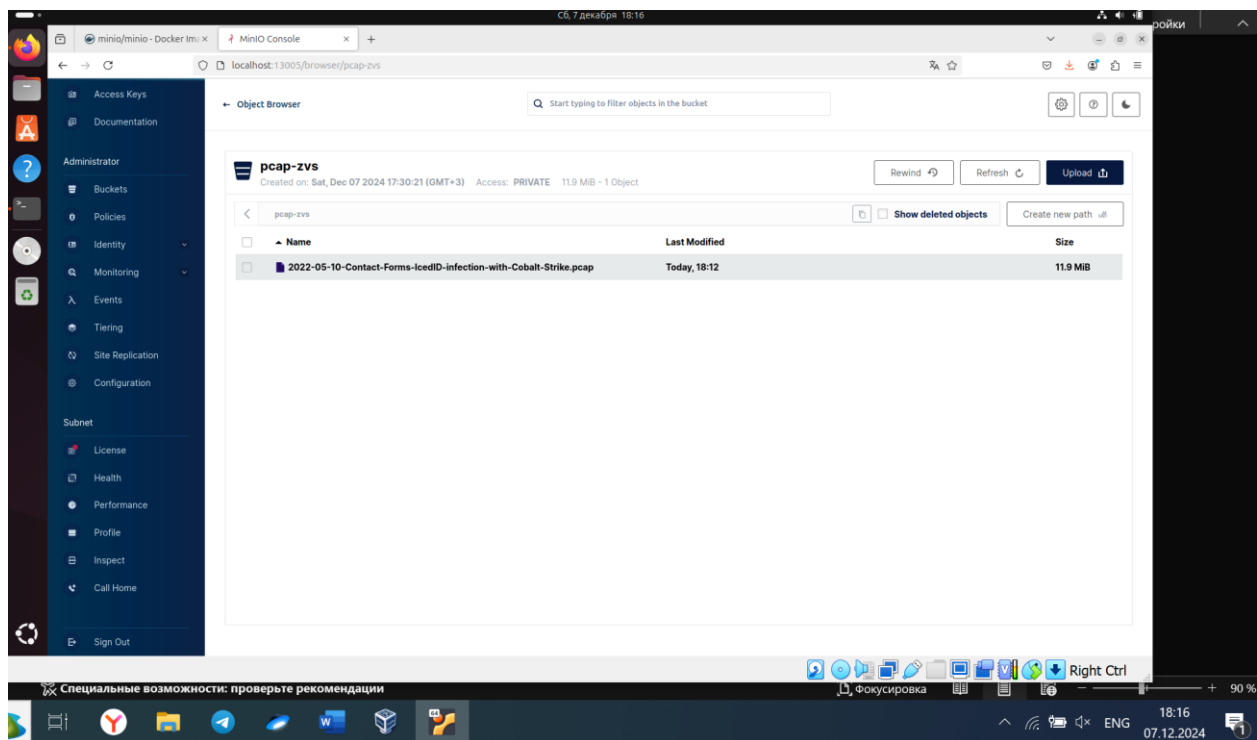




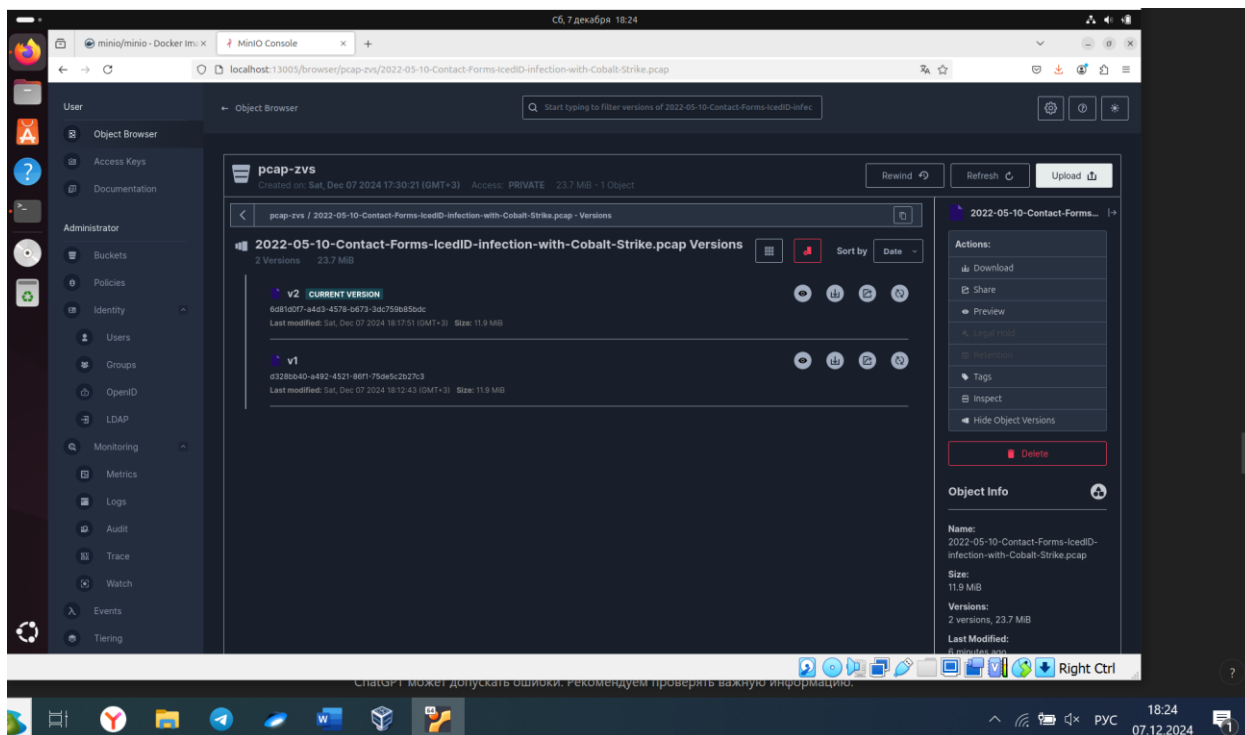
Добавьте разделы (buckets). Названия разделов должны заканчиваться на дефис и инициалы ФИО (например, для Иванова Петра Дмитриевича – rscap-ipd, reports-ipd, и т.п.).



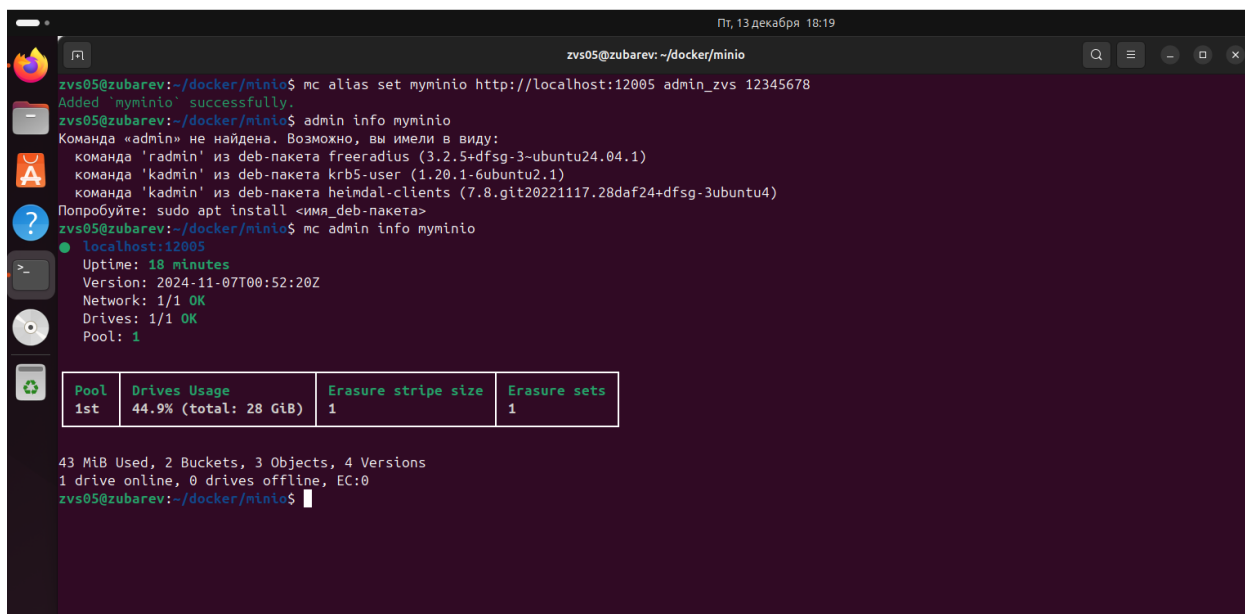
Пример загрузки файлов:



Изучите, как работает версионирование. Для этого загрузите несколько различных версий файла с одинаковым названием в один и тот же раздел (в разделе должно быть включено версионирование).



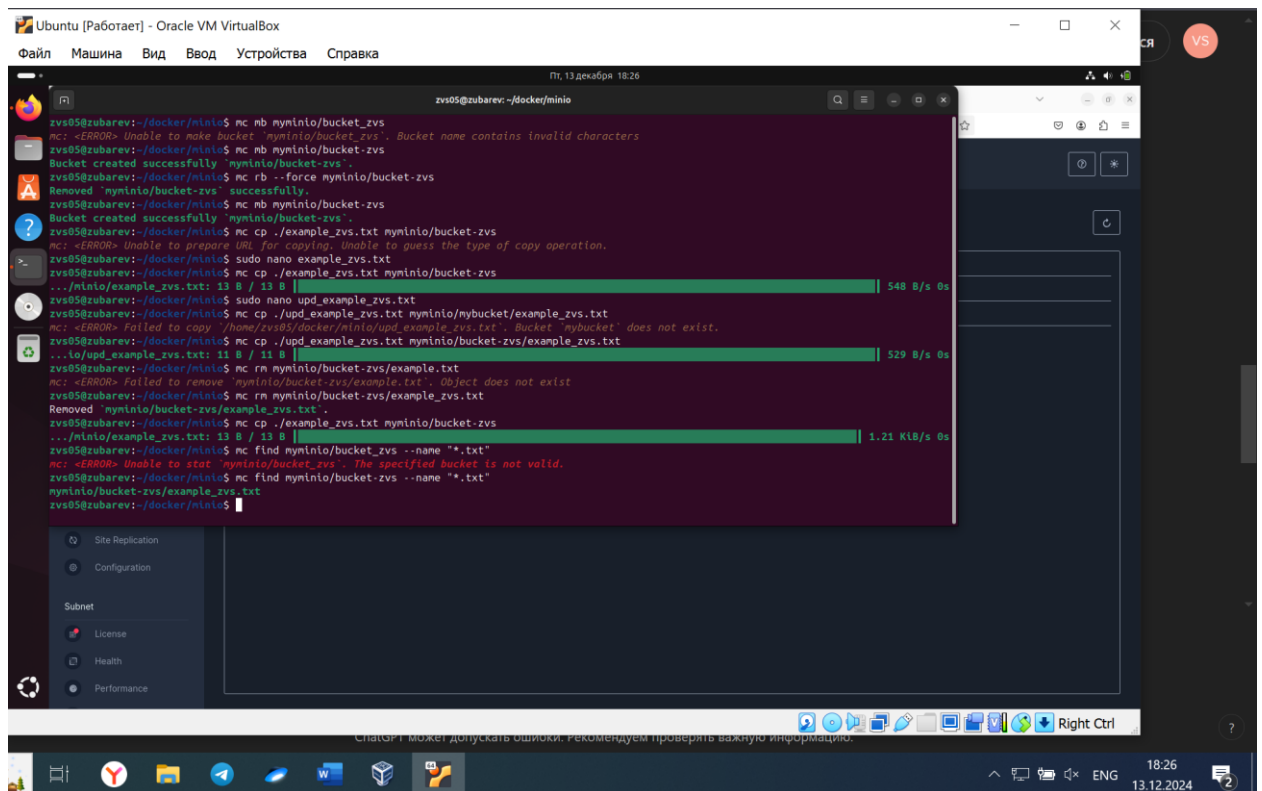
Создайте алиас (alias) в mc (авторизуйтесь).



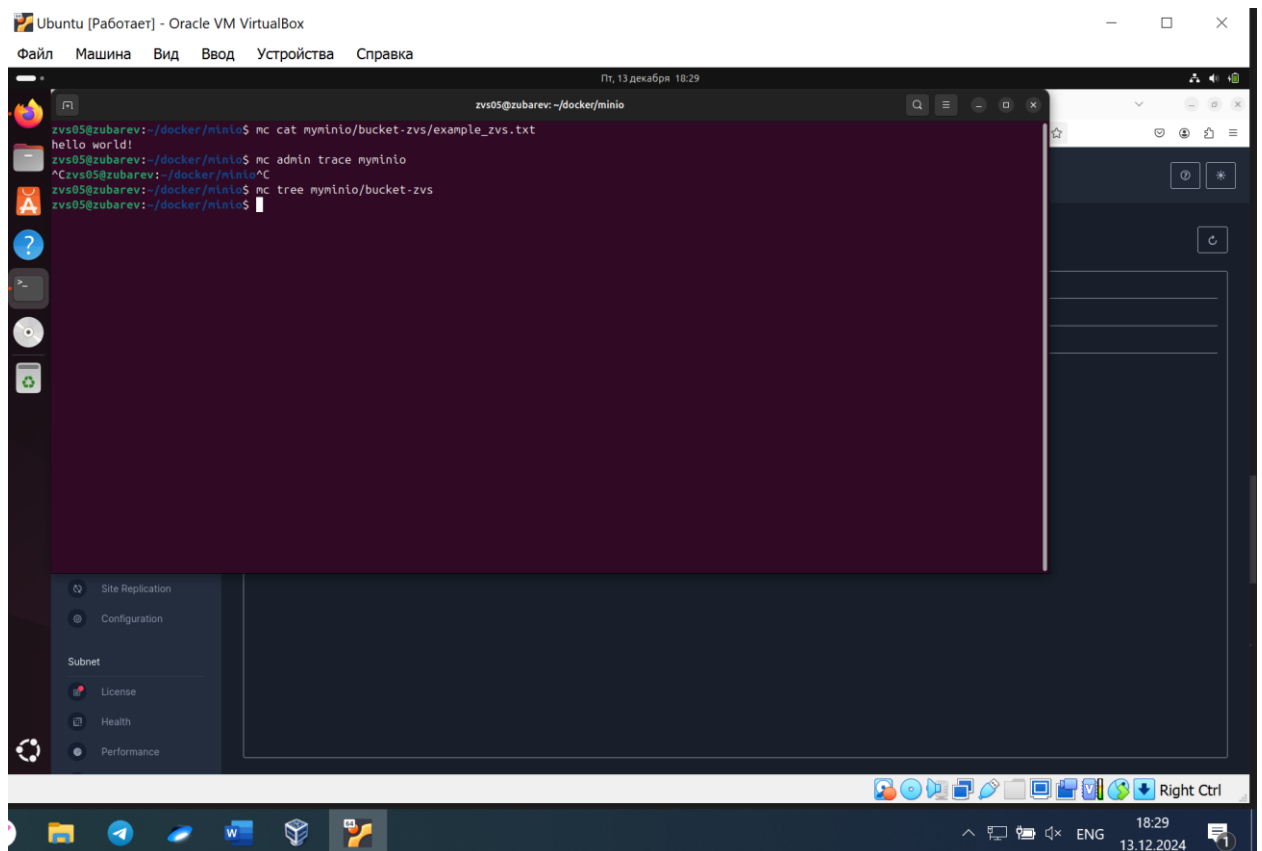
Изучите команды MinIO Client (<https://hub.docker.com/r/minio/mc/>).

Попрактикуйтесь в выполнении основных:

- создание, удаление разделов;
- добавление, обновление, удаление файлов;
- поиск объектов;



- отображение содержимого файлов;
- другие команды (мониторинг действий, вывод структуры проекта).



Напишите скрипт/программу для работы с MinIO. Минимальный функционал:

- создание и удаление разделов;

- загрузка файлов в раздел;
- скачивание файлов;
- удаление файлов;
- просмотр списка файлов в разделе.

Листинг программы:

```
from minio import Minio
from minio.error import S3Error
import os

# Инициализация клиента MinIO
minio_client = Minio(
    "localhost:12005", # Замените на адрес вашего MinIO сервера
    access_key="admin_zvs", # Ваш Access Key
    secret_key="12345678", # Ваш Secret Key
    secure=False
)

# Создание раздела
def create_bucket(bucket_name):
    if not bucket_name.endswith("zvs"):
        print(f"Название раздела должно заканчиваться на 'zvs': {bucket_name}")
        return
    if not minio_client.bucket_exists(bucket_name):
        minio_client.make_bucket(bucket_name)
        print(f"Раздел '{bucket_name}' создан.")
    else:
        print(f"Раздел '{bucket_name}' уже существует.")

# Удаление раздела
def delete_bucket(bucket_name):
    try:
        objects = minio_client.list_objects(bucket_name, recursive=True)
        for obj in objects:
            minio_client.remove_object(bucket_name, obj.object_name)
        minio_client.remove_bucket(bucket_name)
        print(f"Раздел '{bucket_name}' удален.")
    except S3Error as e:
        print(f"Ошибка удаления раздела: {e}")

# Загрузка файла в раздел
def upload_file(bucket_name, file_path):
    if not os.path.exists(file_path):
        print(f"Файл {file_path} не существует.")
        return
    file_name = os.path.basename(file_path)
    if not file_name.startswith("zvs_"):
        print(f"Имя файла должно начинаться с 'zvs_': {file_name}")
        return
    try:
        minio_client.fput_object(bucket_name, file_name, file_path)
        print(f"Файл '{file_name}' загружен в раздел '{bucket_name}'.")
    except S3Error as e:
        print(f"Ошибка загрузки файла: {e}")

# Скачивание файла из раздела
def download_file(bucket_name, object_name, download_path):
    try:
        minio_client.fget_object(bucket_name, object_name, download_path)
```

```

        print(f"Файл '{object_name}' скачан в '{download_path}'")
    except S3Error as e:
        print(f"Ошибка скачивания файла: {e}")

# Удаление файла из раздела
def delete_file(bucket_name, object_name):
    try:
        minio_client.remove_object(bucket_name, object_name)
        print(f"Файл '{object_name}' удален из раздела '{bucket_name}'")
    except S3Error as e:
        print(f"Ошибка удаления файла: {e}")

# Просмотр списка файлов в разделе
def list_files(bucket_name):
    try:
        objects = minio_client.list_objects(bucket_name, recursive=True)
        print(f"Файлы в разделе '{bucket_name}':")
        for obj in objects:
            print(f"- {obj.object_name}")
    except S3Error as e:
        print(f"Ошибка получения списка файлов: {e}")

# Демонстрация работы функций
if __name__ == "__main__":
    bucket_name = "files-zvs"
    create_bucket(bucket_name)

    # Загрузка файлов
    upload_file(bucket_name, "zvs_01.txt") # Файл должен существовать
    upload_file(bucket_name, "zvs_02.txt") # Файл должен существовать

    # Просмотр файлов
    list_files(bucket_name)

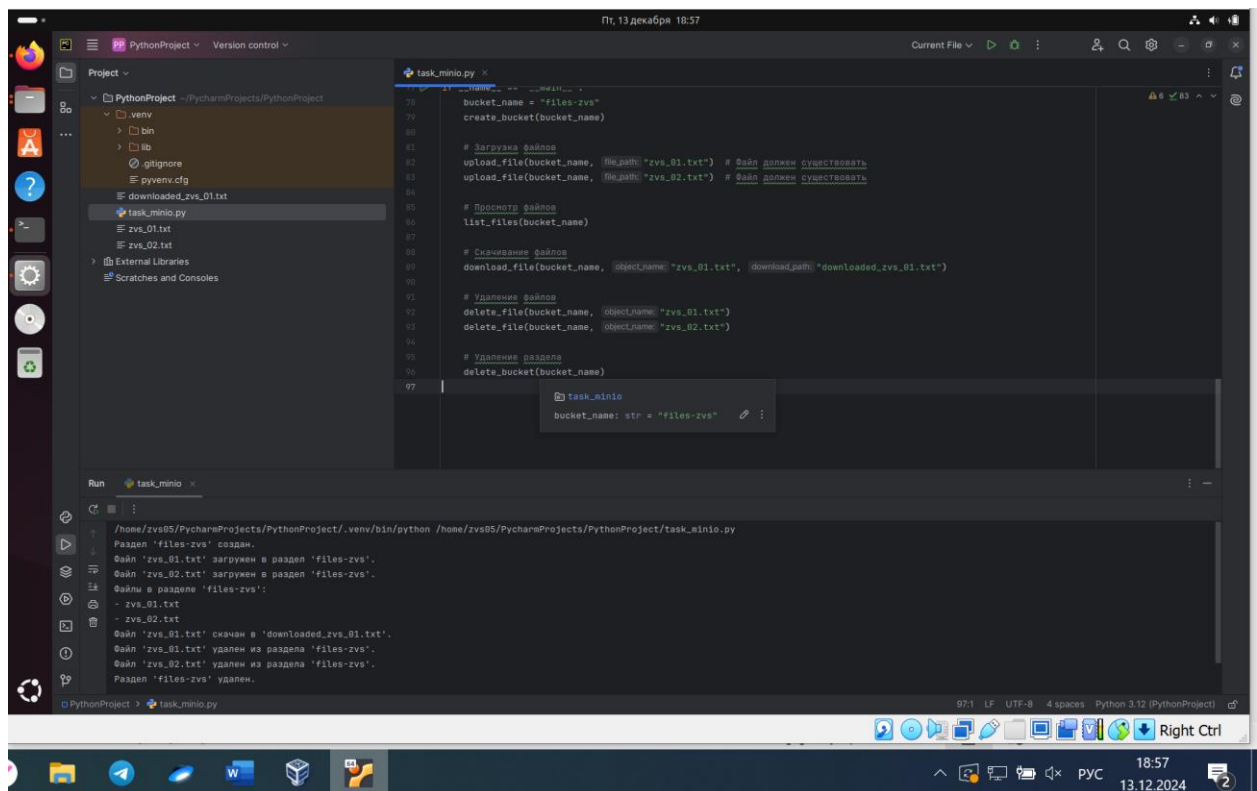
    # Скачивание файлов
    download_file(bucket_name, "zvs_01.txt", "downloaded_zvs_01.txt")

    # Удаление файлов
    delete_file(bucket_name, "zvs_01.txt")
    delete_file(bucket_name, "zvs_02.txt")

    # Удаление раздела
    delete_bucket(bucket_name)

```

Результат работы программы



Разверните MinIO в режиме Multi-Node Multi-Drive (не менее 2 серверов и 4 дисков) и проверьте его работоспособность.

Листинг файла docker-compose:

version: '3.7'

services:

minio1:

image: minio/minio:latest

container_name: minio1_zvs

restart: always

ports:

- "12005:9000"

- "13005:9090"

environment:

MINIO_ROOT_USER: admin_zvs

MINIO_ROOT_PASSWORD: 12345678

command: server http://minio{1...2}/data{1...2} --console-address ":9090"

volumes:

- ./data1:/data1
- ./data2:/data2
- ./config1:/root/.minio

healthcheck:

test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
interval: 30s
timeout: 10s
retries: 3
start_period: 10s

minio2:

image: minio/minio:latest
container_name: minio2_zvs
restart: always

ports:

- "12105:9000"
- "13105:9090"

environment:

MINIO_ROOT_USER: admin_zvs
MINIO_ROOT_PASSWORD: 12345678

command: server http://minio{1...2}/data{1...2} --console-address ":9090"

volumes:

- ./data3:/data1
- ./data4:/data2
- ./config2:/root/.minio

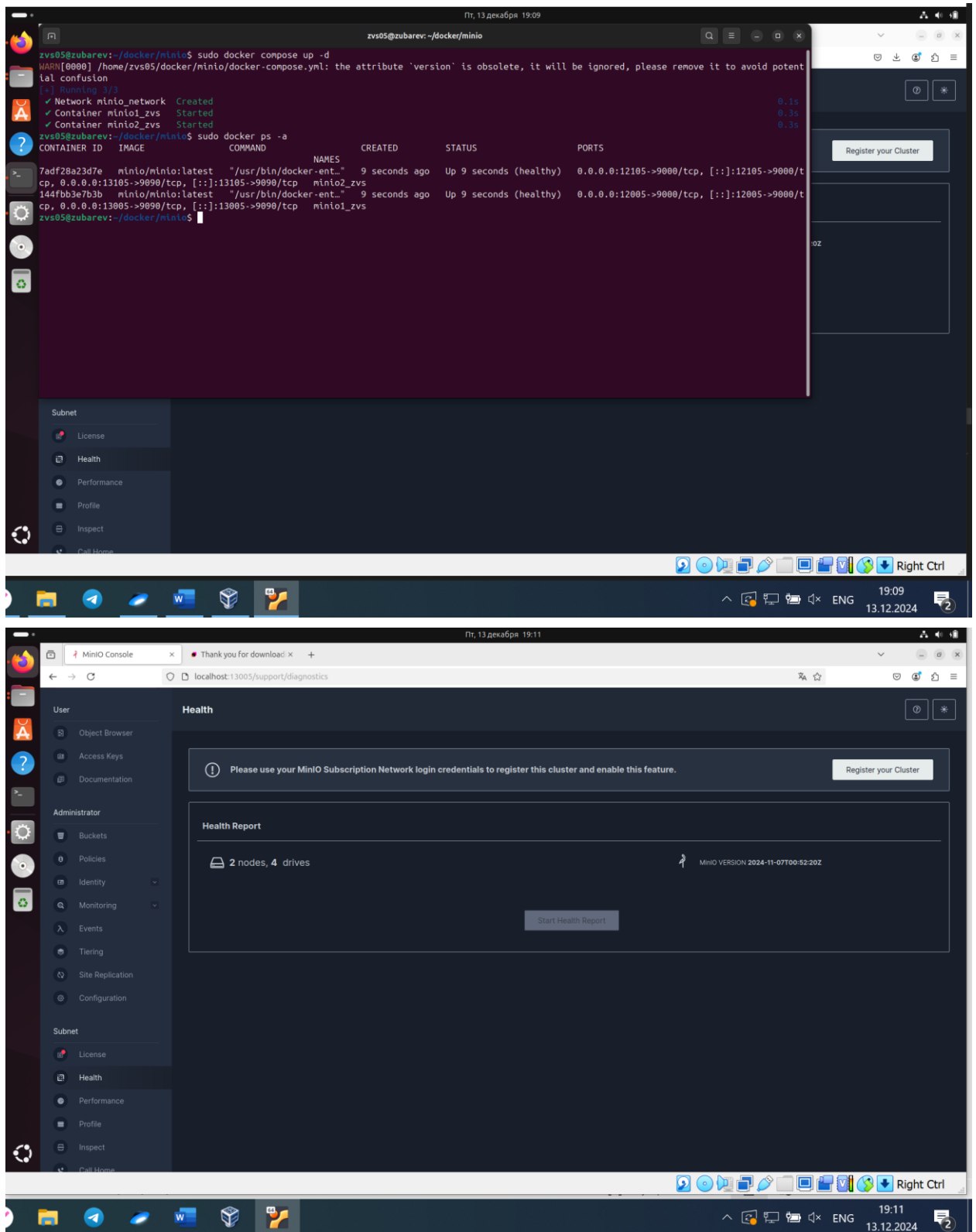
healthcheck:

test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
interval: 30s
timeout: 10s
retries: 3
start_period: 10s

networks:

default:

name: minio_network



Создайте каталог для проекта и сформируйте файл `docker-compose.yml` для разворачивания Neo4j (https://hub.docker.com/_/neo4j).

Листинг файла docker-compose

version: '3.7'

services:

neo4j:

image: neo4j:latest

container_name: neo4j_zvs

restart: always

ports:

- "7405:7474" # Порт для веб-интерфейса Neo4j

- "7605:7687" # Порт для протокола Volt

environment:

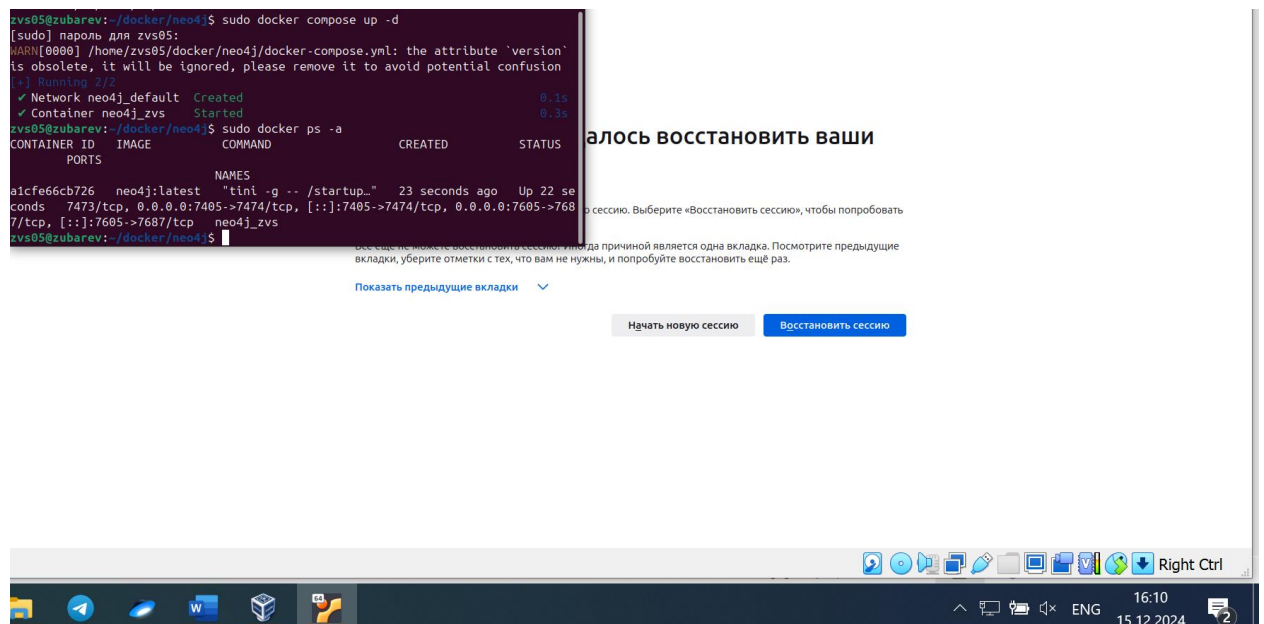
NEO4J_AUTH: none

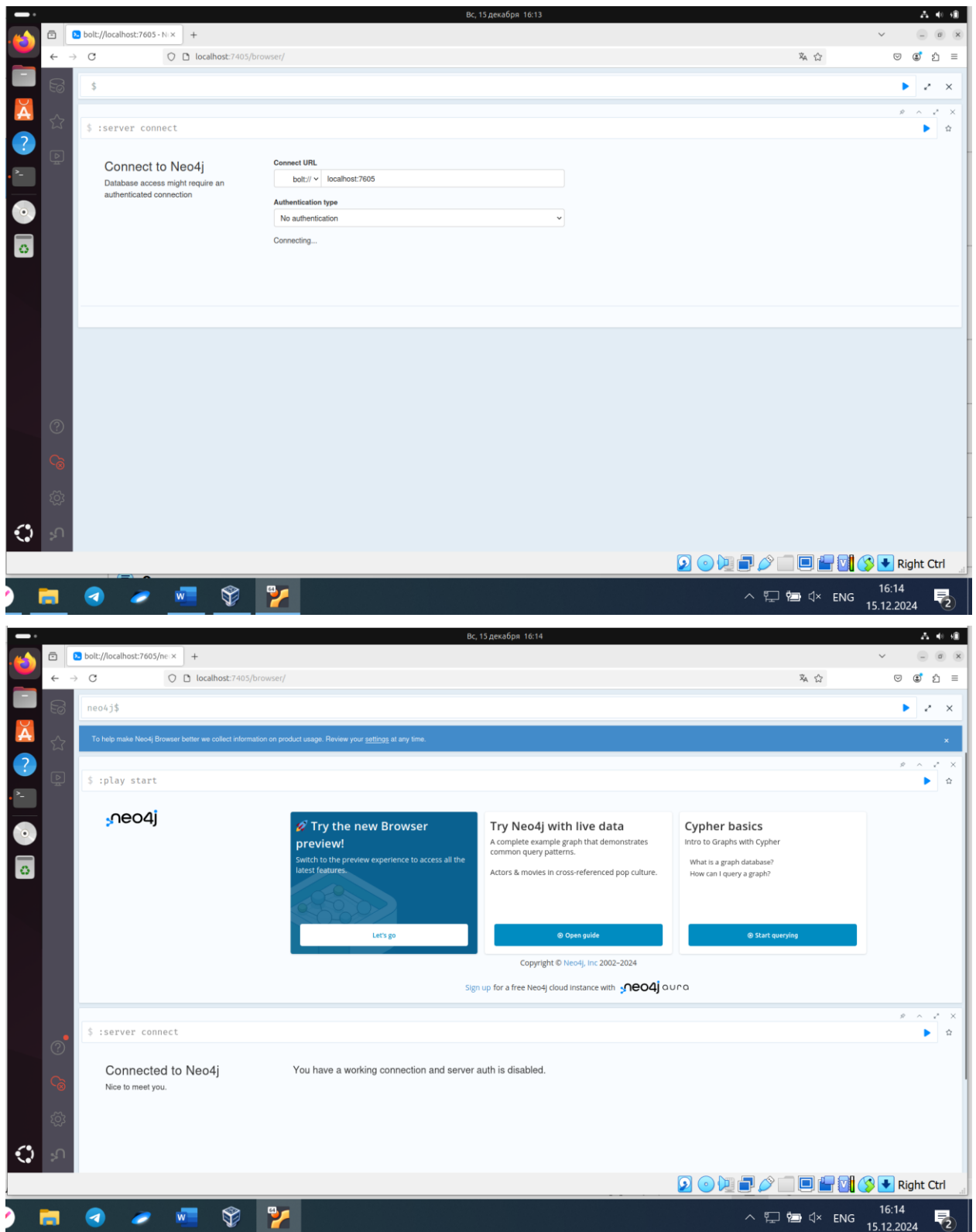
NEO4J_dbms_directories_import: /import

volumes:

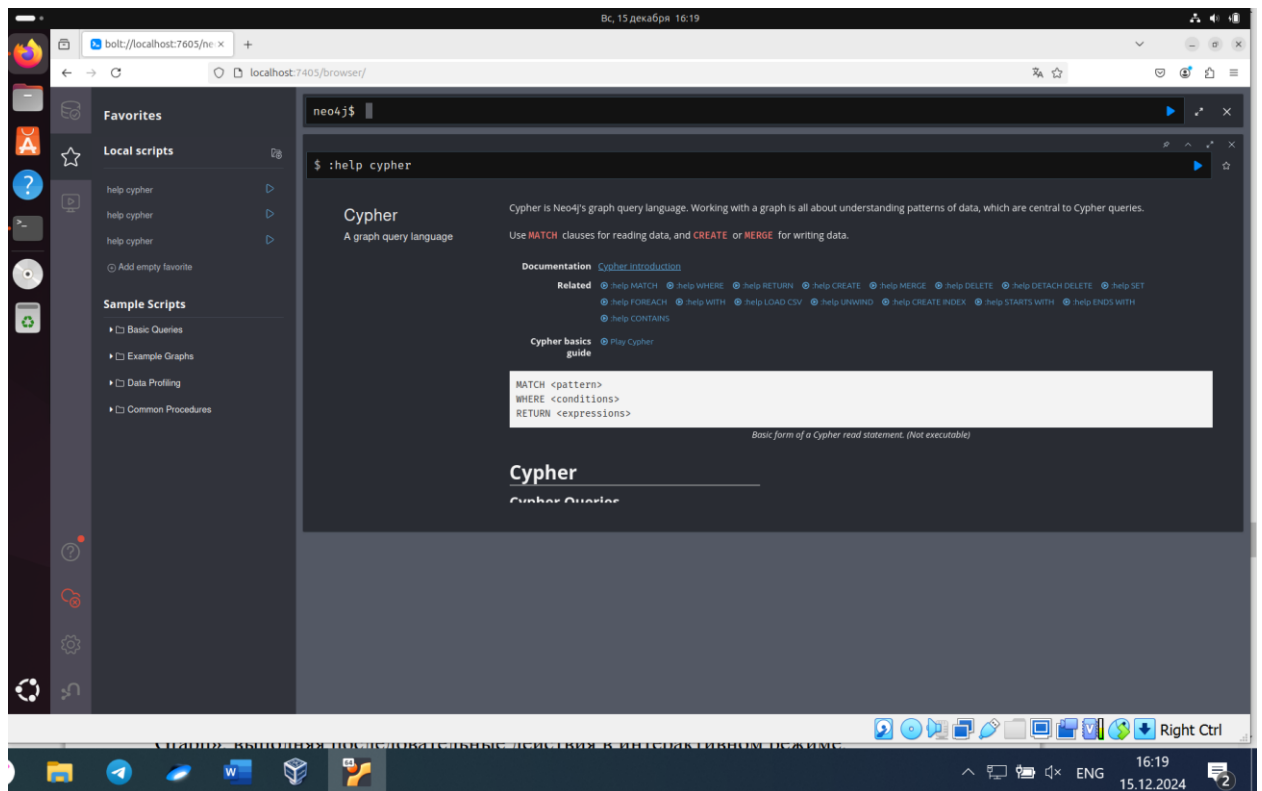
- ./data:/data

- ./import:/import

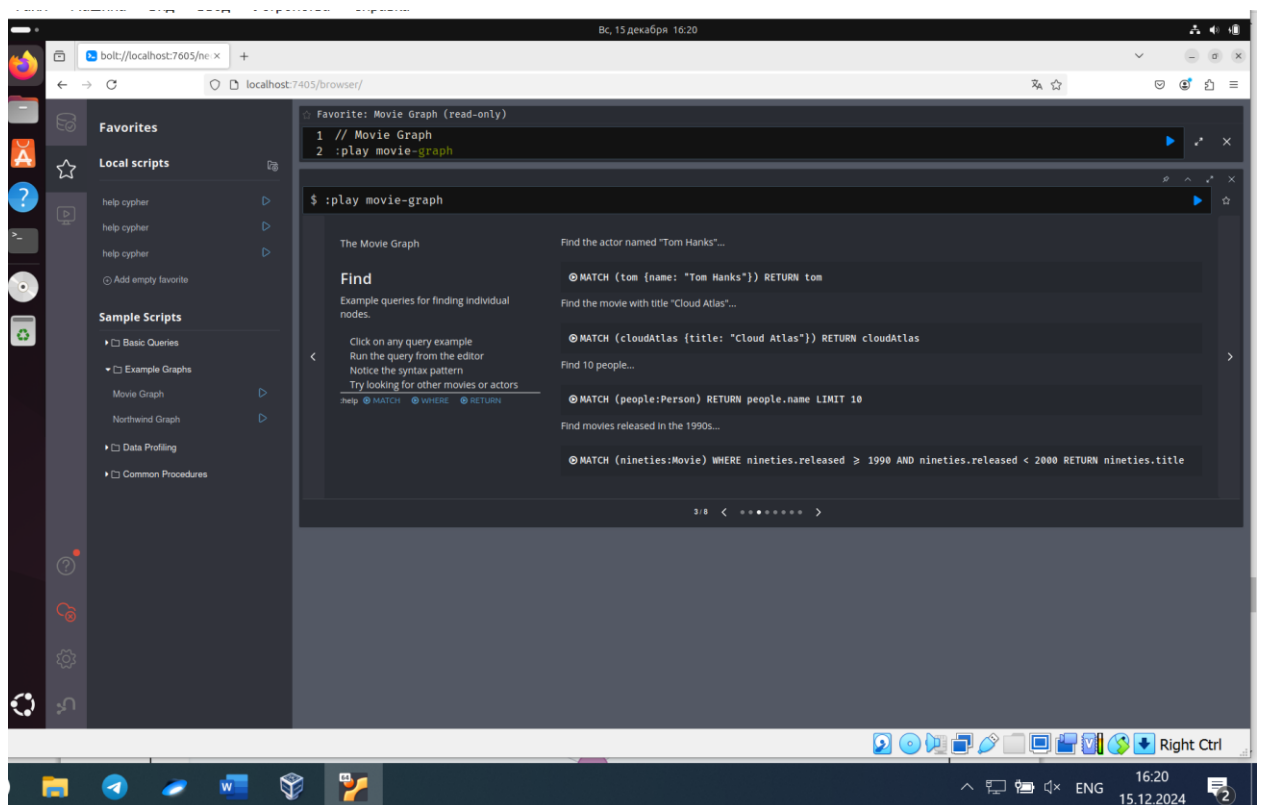


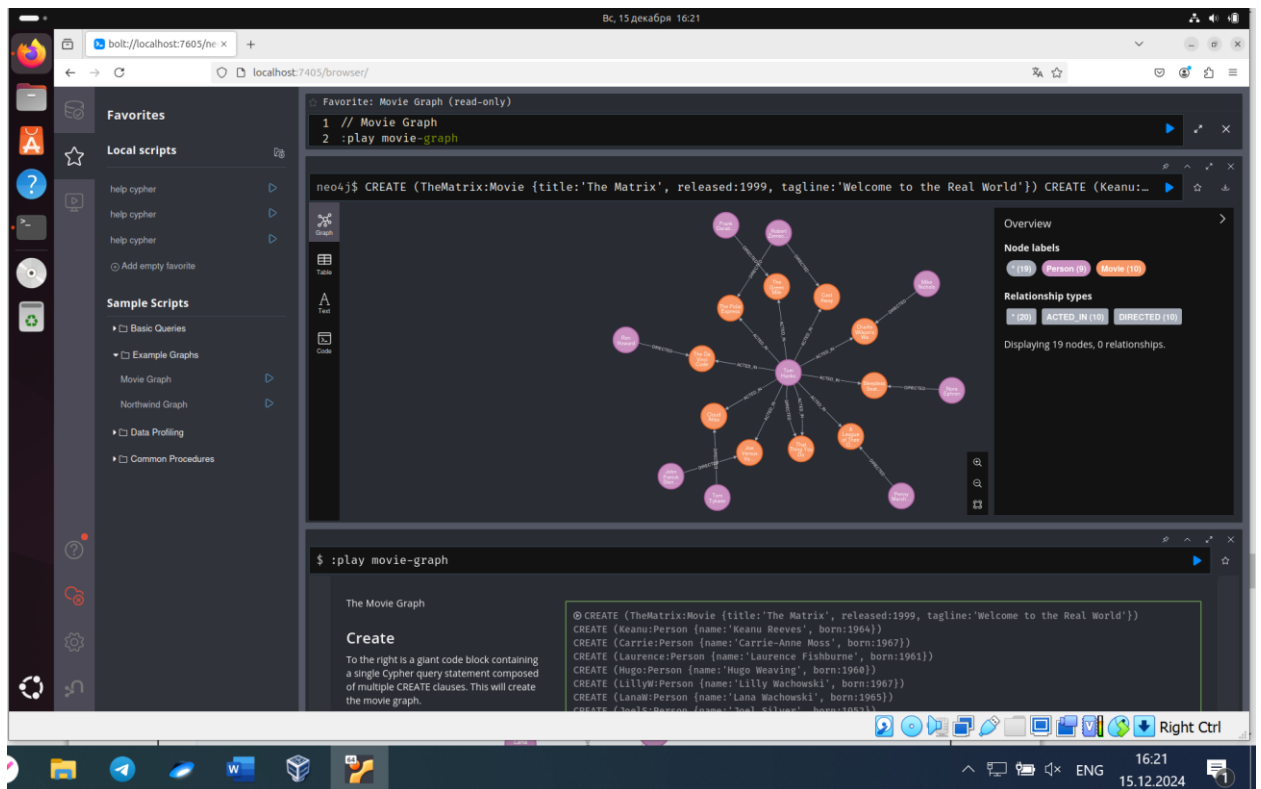


Изучите синтаксис написания запросов Cypher Query Language с использованием команды «:help cypher»

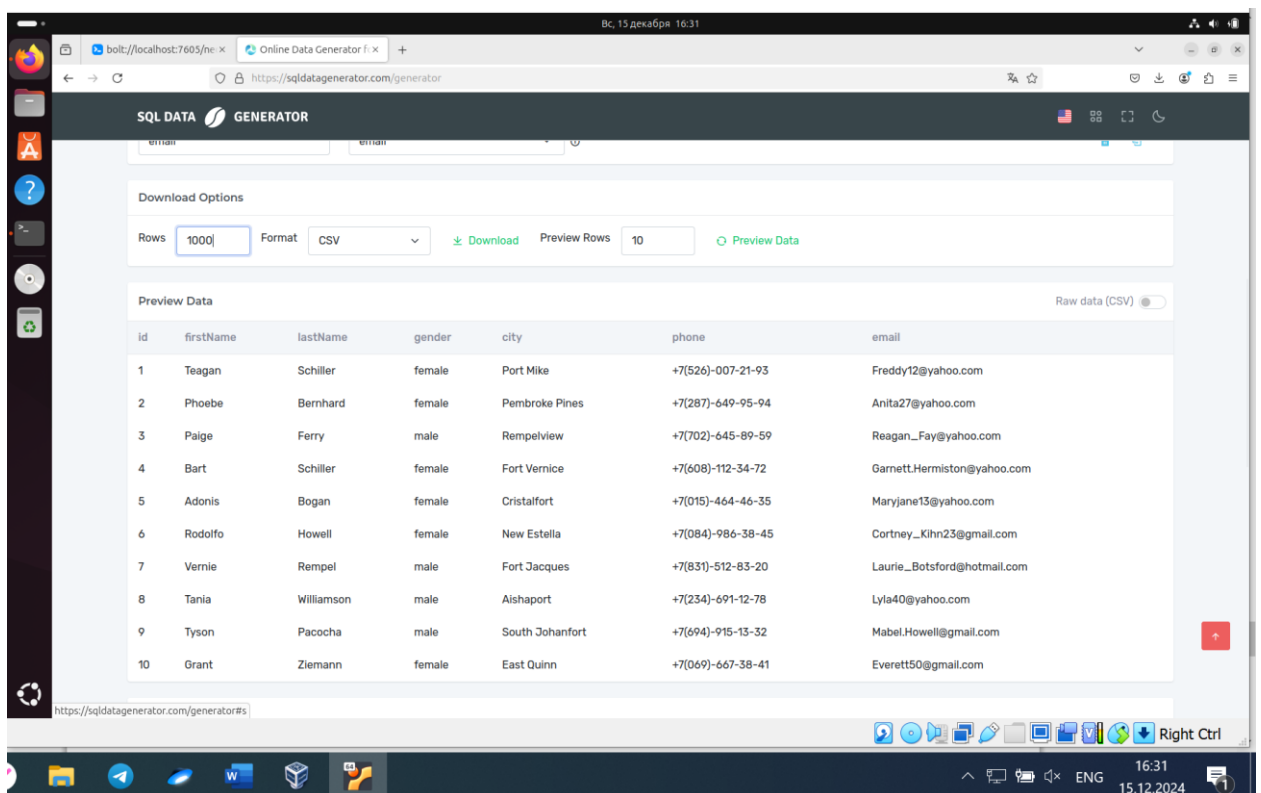


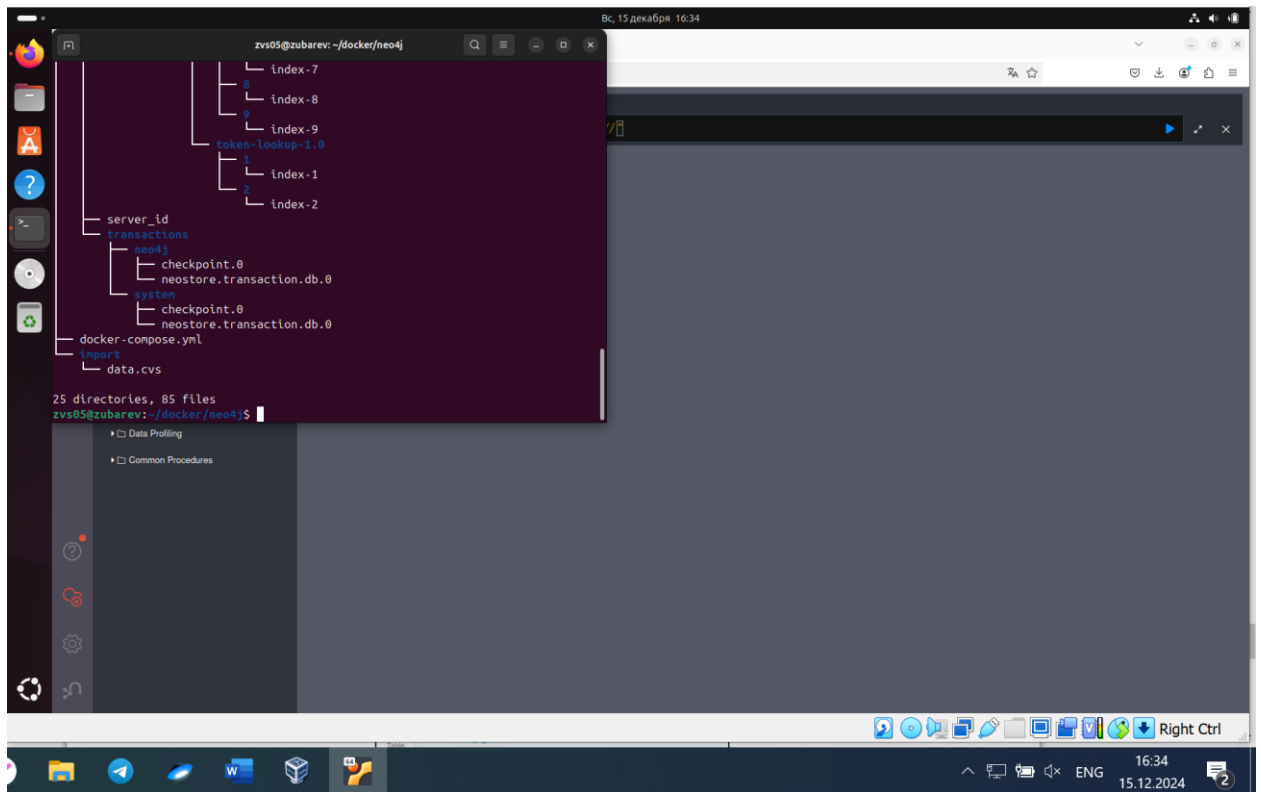
Изучите основные операции и возможности в Neo4j на примере графа «Movie Graph», выполняя последовательные действия в интерактивном режиме.



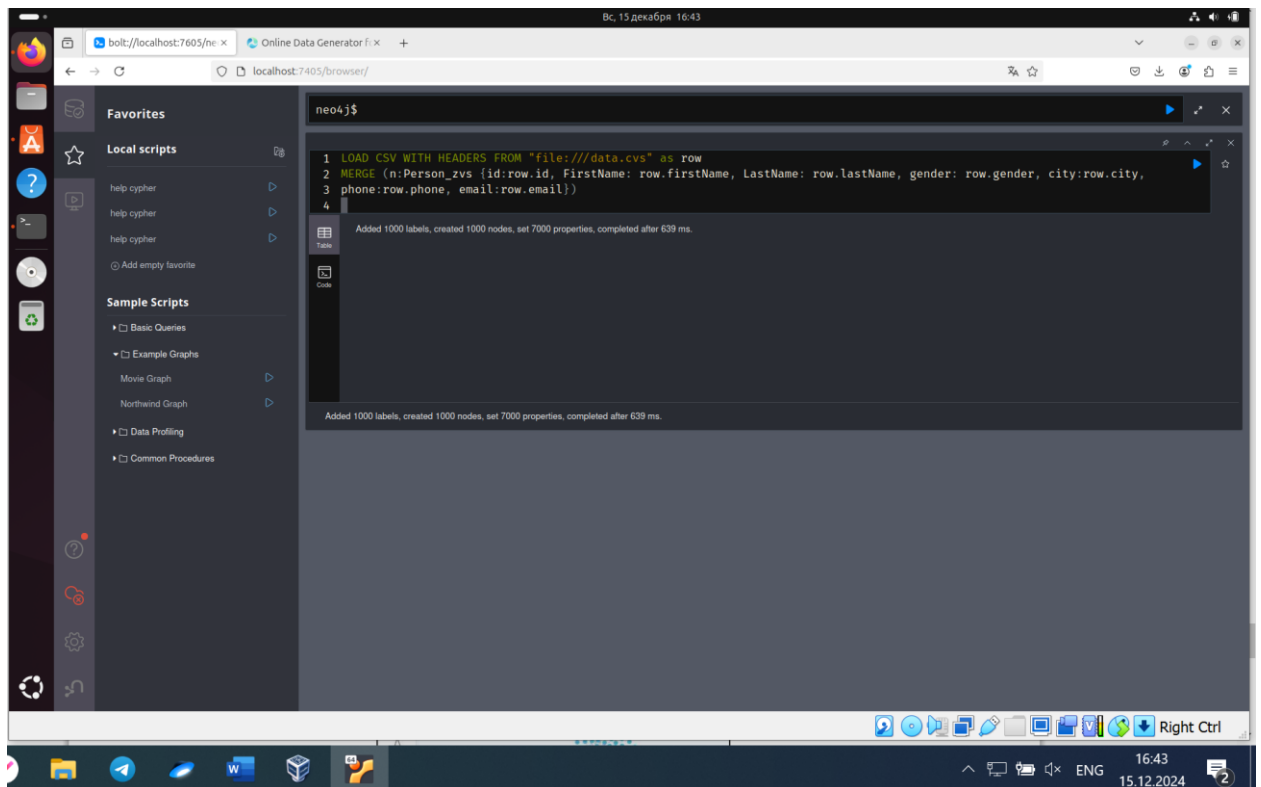


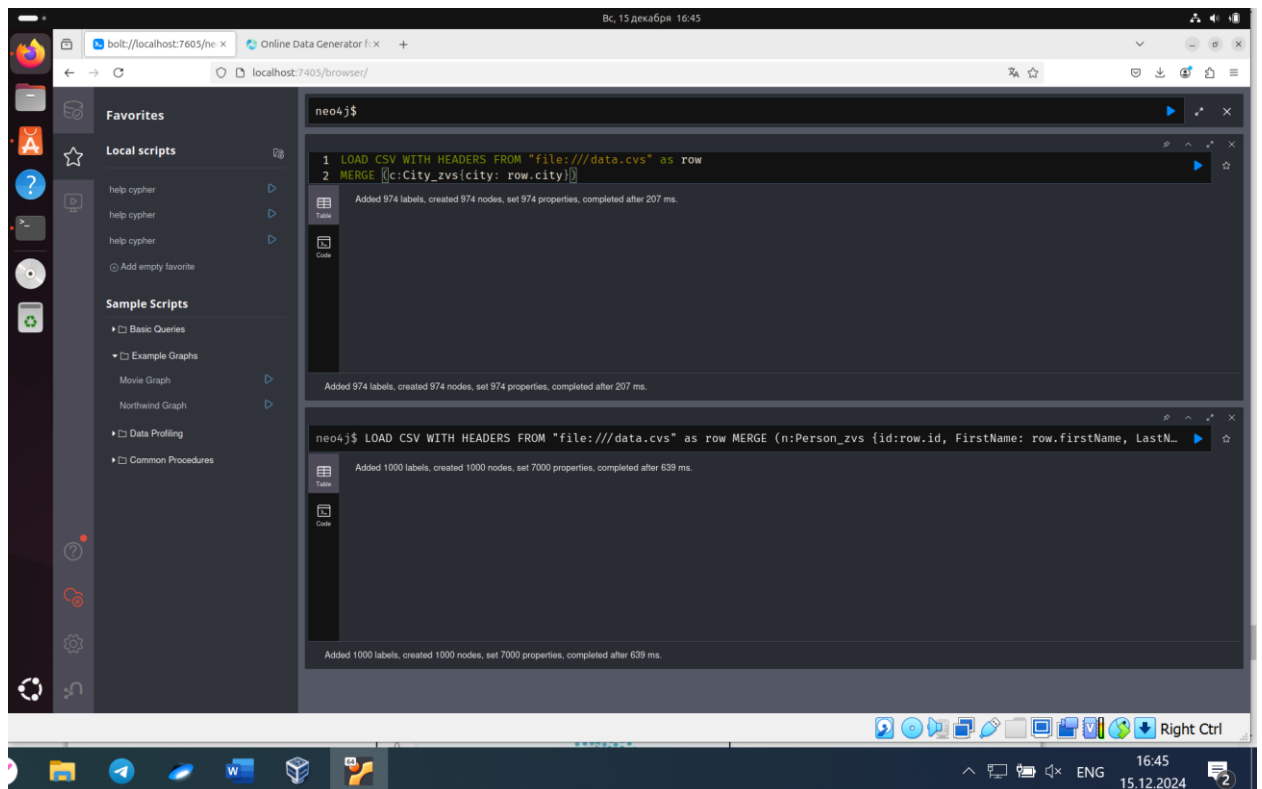
Сформируйте тестовый набор данных в формате CSV для импорта и создания графа в Neo4j (например, на сайте <https://sqldatagenerator.com/generator>). Поместите файл в каталог import



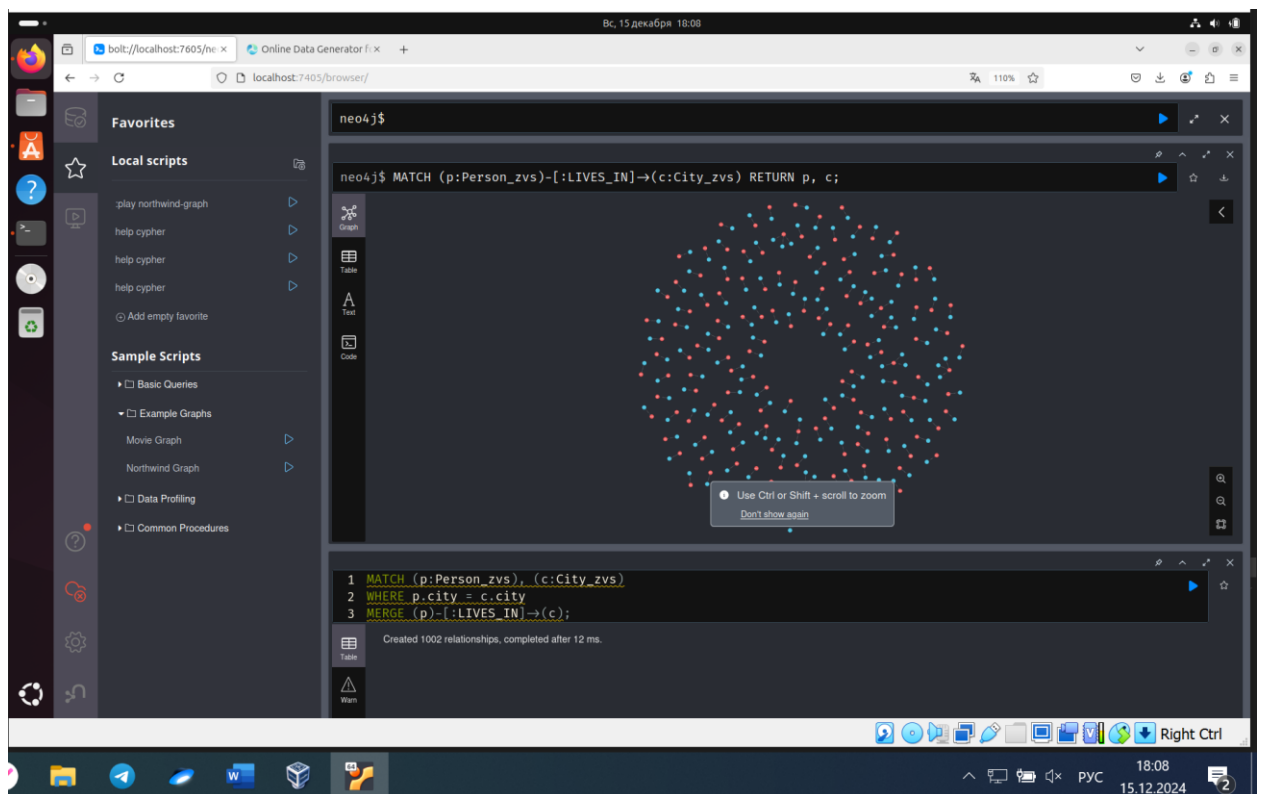


Импортируйте данные в Neo4j.





Создайте связи между двумя сущностями
Проанализируйте полученный граф.



Граф показывает какие люди живут в каких городах

Выполните в созданном графе операции:

-добавления узлов

The screenshot displays the Neo4j desktop application. The top panel shows a Cypher query editor with the following code:

```
1 CREATE (p1:Person_zvs {name: 'Vasiliy', age: 20})
2 CREATE (p2:Person_zvs {name: 'Andrey', age: 20})
3 RETURN p1, p2;
```

The central area shows a graph visualization with two blue circular nodes and a connecting line. The right panel, titled 'Node properties', shows the details for a selected node of type 'Person_zvs':

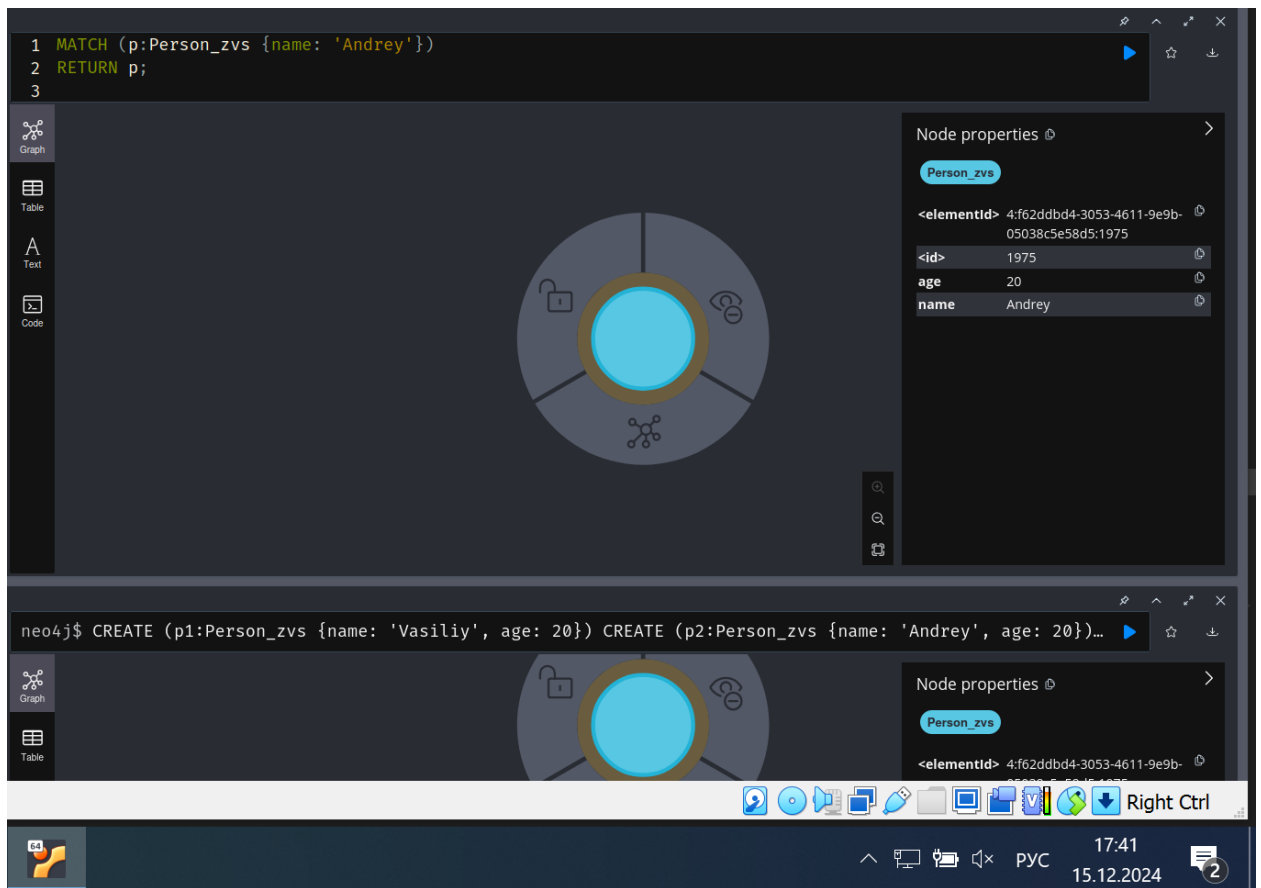
Property	Value
<elementId>	4:f62ddb4-3053-4611-9e9b-05038c5e58d5:1975
<id>	1975
age	20
name	Andrey

The bottom panel shows a command prompt with the following command and its output:

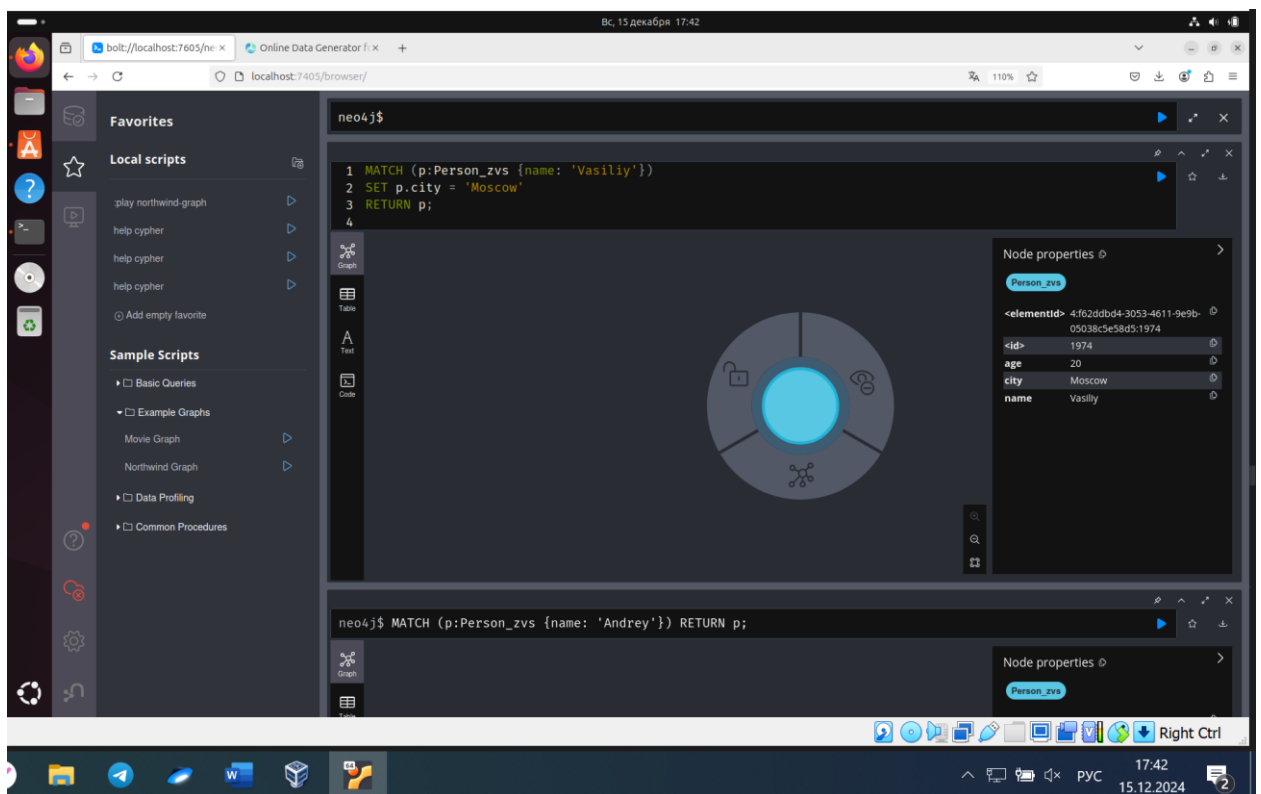
```
neo4j$ LOAD CSV WITH HEADERS FROM "file:///data.csv" as row MERGE (c:City_zvs{city:row.city})
```

Output: Added 974 labels, created 974 nodes, set 974 properties, completed after 220 ms.

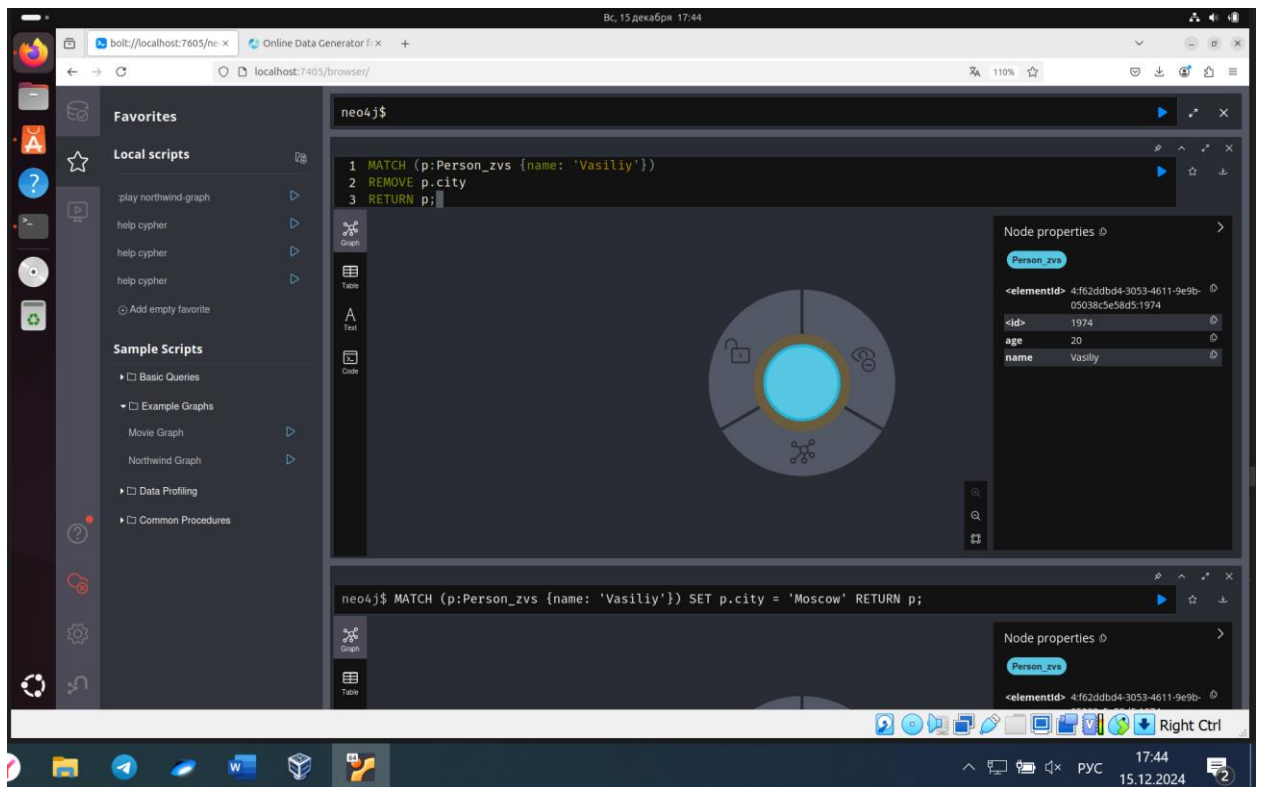
-поиска узлов



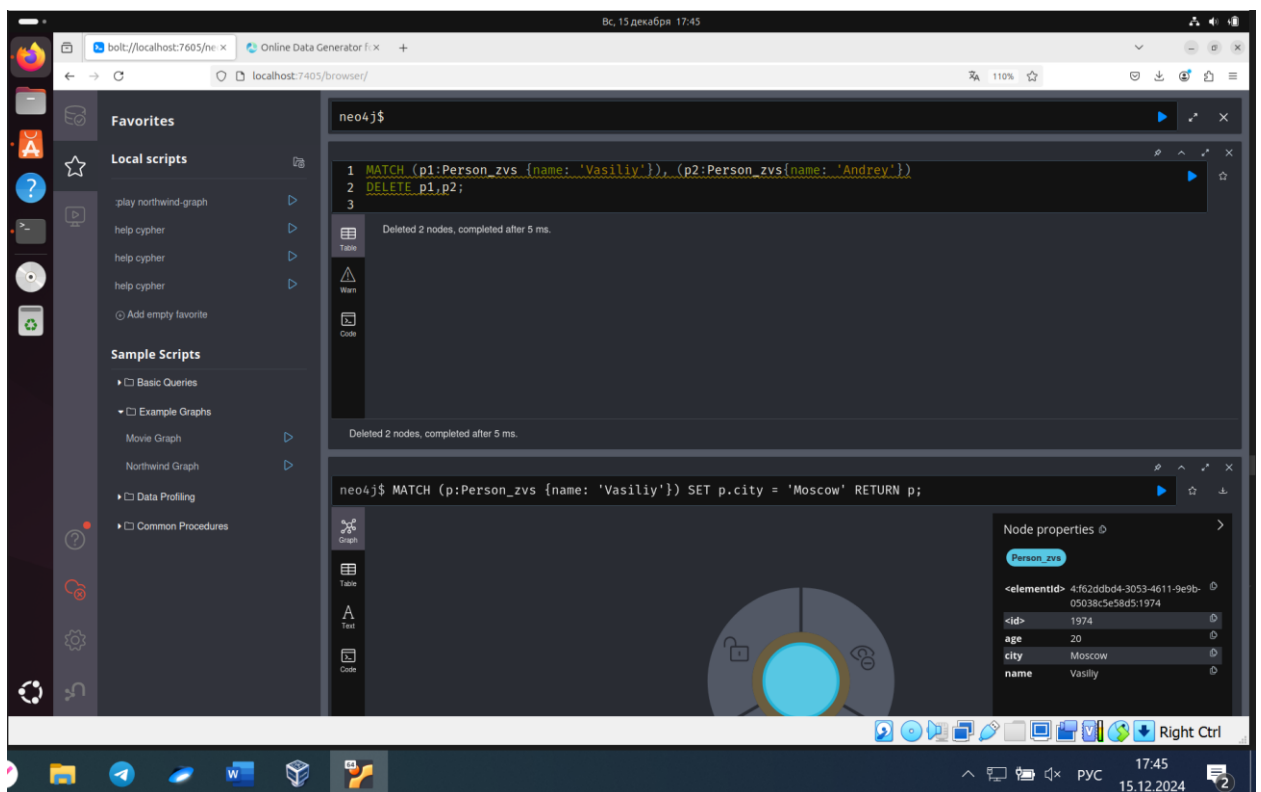
—добавления свойств



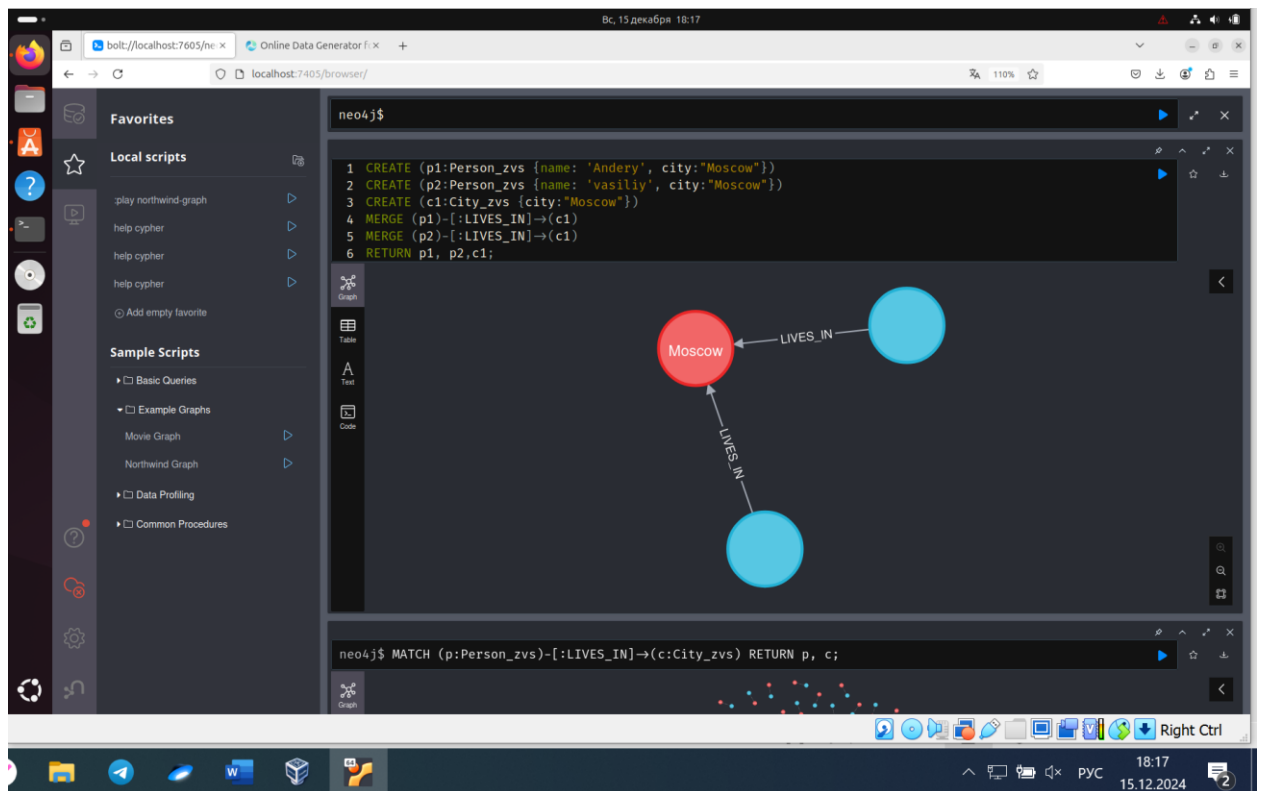
—удаления свойств.



-удаления узлов;



Добавьте новые сущности и атрибуты в ваш граф. Сформируйте новые связи.
Проанализируйте полученные группы объектов.



Теперь есть новый узел с меткой City_zvs и атрибутом city="Moscow", а также два узла с меткой Person_zvs у которых атрибут также атрибут city="Moscow". Они имеют связь между собой по атрибуту city